University of Stavanger

ALEKSANDER BØ, BASEL FERAS ABU SABLIH AND VEBJØRN ERLANDSEN
DEPARTMENT OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Simulation-Based Testing Framework for Embedded Systems Using MQTT and Cross-Compiled Code

Bachelor's Thesis - Computer Science & Electrical Engineering - November 2026

Erklæring om bruk av KI-verktøy ved oppgaveinnlevering

Navn: Aleksander Bø, Basel Feras Abu Sablih og Vebjørn Erlandsen

Emnekode- og navn: ELEBAC - Bacheloroppgave i elektroteknologi / DAT-BAC Bacheloroppgave i datateknologi

Semester: Vår 2026 - 6. semester

Ved UiS er det tillatt å bruke KI-baserte verktøy om ikke annet fremgår av emnebeskrivelsen eller oppgaven.

Riktig bruk av KI-verktøy vil som hovedprinsipp være å benytte KI-verktøyet som et hjelpemiddel, men hvor besvarelsen utarbeides av studentene selv og hvor kilder oppgis i tråd med regelverket. Bachelor- og masteroppgaver, samt andre innleveringsoppgaver er skriftlige og selvstendige oppgaver. Det er du som er ansvarlig for den endelige teksten som leveres inn til sensur.

For mer informasjon om riktig bruk av KI-verktøy:
Eksamen | Universitetet i Stavanger

Kryss av i skjemaet for hvordan du har brukt/ikke brukt KI-verktøy i din oppgave.

Målet med å beskrive bruken er å vise leseren hvordan du har gått fram i arbeidet med oppgaven din. Med dette følger du kravene om åpenhet og redelighet i en akademisk oppgave, og det kan gjøre det enklere for sensoren å vurdere ditt eget bidrag. Gjennom å være tydelig på hvordan du gikk fram, vil sensoren få en forklaring på hvordan du har brukt språkmodeller i din oppgave.

Språkvask

Har du brukt generativ kunstig intelligens til språkvask/korrektur av deler eller hele teksten?

- ☒ Ja
- ☐ Nei

Hvis ja, spesifiser hvilke verktøy du har brukt og beskriv kort hvordan du har brukt det i oppgaven:

Claude ai. Matet med rå tekst og hjulpet med skrivefeil og akademisk skriving. Overleaf sin AI Writefull for skrivefeil.

Idémyldring

Har du brukt generativ kunstig intelligens til ideer og forslag til struktur i oppgaven din?

- ☒ Ja
- ☐ Nei

Hvis ja, spesifiser hvilke verktøy du har brukt og hvordan du har brukt det i oppgaven:

Claude ai. Generert forslag til rekkefølge på kapitler.

Bildegenerering

Har du generert ett eller flere av bildene/figurene i oppgaven/rapporten med generativ kunstig intelligens?

☐ Ja

☒ Nei

☐ Ikke relevant

Hvis ja, spesifiser hvilke verktøy du har brukt og hvordan du har brukt det i oppgaven:

Programmeringshjelp

Har du generert deler av koden/algoritmene som

1. framkommer direkte i rapporten/avhandlingen/oppgaven eller
2. har blitt brukt til å produsere resultater som figurer, tabeller eller numeriske verdier

med generativ kunstig intelligens?

- ☒ Ja
- ☐ Nei
- ☐ Ikke relevant

Hvis ja, spesifiser hvilke verktøy du har brukt og hvordan du har brukt det i oppgaven: Claude AI har blitt brukt for å rette opp i feilmeldinger og gi forslag til bedre funksjoner om det fantes, men koden er skrevet selv.

Annet

Har du brukt generativ kunstig intelligens på andre måter i rapporten/oppgaven?

- ☐ Ja
- ☒ Nei

Hvis ja, spesifiser hvilke verktøy du har brukt og hvordan du har brukt det i oppgaven:

- ☒ Jeg bekrefter å ha lest informasjon om bruk av KI-verktøy i forbindelse med oppgaveskriving og eksamen.

We, **Aleksander Bø, Basel Feras Abu Sablih and Vebjørn Erlandsen**, declare that this thesis titled, “Simulation-Based Testing Framework for Embedded Systems Using MQTT and Cross-Compiled Code ” and the work presented in it are our own. We confirm that:

- This work was done wholly or mainly while in candidature for a bachelor’s degree at the University of Stavanger.
- Where we have consulted the published work of others, this is always clearly attributed.
- Where we have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely our own work.
- We have acknowledged all main sources of help.

Abstract

This thesis presents a simulation framework that connects the Virtual IPS controller (ABB Integrated Paint System controller) and simulation components using MQTT (Message Queuing Telemetry Transport), a lightweight publish/subscribe messaging protocol, and Windows shared memory. The framework works together with four components: the IPS controller, a proxy that reads analog output registers from shared memory and publishes setpoint values over MQTT, a message broker that routes signals between components, and a RLC circuit simulator that models a second-order electrical system and returns feedback to the controller.

The RLC circuit was selected as the simulated model because its second-order dynamics that are representative of real industrial processes.

The results indicate that MQTT-based communication is feasible for virtual embedded systems testing, though further work is needed to validate simulation accuracy and integrate the framework with ABB RobotStudio, their programming and simulation tool for robotic applications.

Acknowledgements

We are grateful to our supervisor, Morten Mossige, for his enthusiasm and support in writing this thesis, and to ABB Bryne for providing office space to work at, and ensuring we had proper food for lunch and coffee to keep us concentrated.

Contents

Abstract vi

Acknowledgements vii

1 Introduction 1

1.1 About ABB 1

1.1.1 ABB BRYNE (1941) 1

1.2 Background and motivation 2

1.2.1 Background 2

1.2.2 Problem Statement 3

1.2.3 Framework Requirements 3

1.2.4 Proposed solution 4

1.2.5 Delimitations 5

1.2.6 Research Questions 5

1.2.7 Goals 6

1.2.8 Choice of Simulated Process 6

1.3 Outline 8

2 Technology **9**

2.1 Programming languages 9

2.1.1 Python 9

2.1.2 C# 9

2.2 Protocol 10

2.2.1 MQTT 10

2.3 Frameworks 11

2.3.1 Python paho-MQTT 12

2.3.2 C# MQTTnet 12

2.4 Data formats 12

2.4.1 JSON 12

2.5 Development tools and runtime environments 12

2.5.1 ABB RobotStudio 13

2.5.2 Tera Term 13

3 Hardware **14**

3.1 IPS APIP-04 14

3.2 IPS Hardware Interface 16

3.2.1 Analog Input and Output Handling 16

3.2.2 Terminal interface and commands 16

3.3 Representing an industrial process 17

3.3.1 RLC Circuit as Representative Industrial Process 17

3.3.2 Component Selection and Transfer Function 18

3.4 Physical Setup 20

3.4.1 Test Procedure 22
3.4.2 Measured Response 23

4 Framework design **24**

4.1 Components 26
4.1.1 ABB CPU Controller 26
4.1.2 Proxy 30
4.1.3 MQTT Broker 32

5 RLC Circuit Model **34**

5.1 Introduction 34
5.1.1 Relation to the Simulation Framework 34
5.2 Mathematical Model 35
5.3 Numerical Simulation 36
5.4 Python Implementation 37
5.4.1 Receiving data 38
5.4.2 Main loop 39
5.5 Simulation Results 40

6 Testing and Validation **41**

6.1 Latency 41
6.1.1 Round-trip time 41
6.2 Accuracy 44
6.2.1 AI1 Channel Verification 44
6.2.2 Hardware-Simulator Comparison Method 45

6.2.3 Results 45
6.3 Robustness 48
6.3.1 MQTT Broker failure handling 48
6.3.2 Limitations in Robustness Testing 50

7 Discussion **52**
7.1 Simulation Realism 52
7.2 Limitations 53
7.2.1 Broker dependency 53
7.2.2 Platform dependency 53
7.2.3 Simplified simulator model 53
7.2.4 Asynchronous execution 53
7.2.5 Substitution of motor model with RLC circuit 53
7.3 Further improvements 54
7.3.1 Integration with ABB RobotStudio 54
7.3.2 More realistic simulation Models 55
7.3.3 Integration with External Simulation Tools 55
7.3.4 Timing and Synchronization Improvements 55
7.3.5 Monitoring and Testing 56

8 Conclusion **57**

A IPS Reference **59**
A.1 Communication and Terminal Interface 59
A.1.1 Serial Communication Setup 59

A.1.2 Configuration File 60
A.1.3 Analog Input and Output via Terminal 61

Chapter 1

Introduction

1.1 About ABB

ABB is a global technology company specializing in electrification and automation, with approximately 110000 employees worldwide. The company develops a broad range of products and systems including industrial robots, motor drives, switchgear, and process control systems. ABB's stated purpose is to enable a more sustainable and resource-efficient future through its technology and industry leadership. [2]

1.1.1 ABB BRYNE (1941)

[1] ABB Bryne is one of ABB's research and development sites, located in Bryne, Norway. ABB Bryne created the first painting robot in 1969. Today, robots are manufactured elsewhere, but ABB Bryne is still active doing research and improving the ABB painting robot technology.



Figure 1.1: ABB Bryne's first painting robot from 1969. [1]

1.2 Background and motivation

This section provides the necessary background, that defines the problem addressed in this thesis, and presents the research questions and goals guiding the work.

1.2.1 Background

In industrial automation, testing control systems typically requires physical hardware such as sensors, actuators, and embedded controllers. However, such hardware is often expensive and not available during early development. Simulation-based testing therefore plays a key role in validating system behavior before deployment.

IPS (Integrated Painting System) is ABB's control system for painting equipment,

responsible for low-level I/O, signal handling, and process regulation. The system runs on the APIP-04, a dedicated embedded controller developed by ABB, which provides analog and digital I/O interfaces for interacting with sensors and actuators in the painting process.

The IPS software can be compiled for both the physical APIP-04 hardware target and for windows, producing what is referred to as a Virtual IPS, a software-only instance that replaces hardware I/O with in-memory abstractions, enabling development and testing without physical devices.

Existing virtual environments for IPS lack realistic low-level I/O behavior and plant interaction, limiting their ability to support closed-loop testing. This thesis proposes and implements a simulation framework where the controller interacts with a virtual representation of the physical hardware through a communication layer, enabling realistic closed-loop testing without physical hardware.

1.2.2 Problem Statement

Although a Virtual IPS exists, it currently has no mechanism for interacting with a simulated physical process. When the controller writes a motor setpoint or opens a valve, there is no simulated plant to respond, no feedback is returned to the controller. This prevents closed-loop validation, where the correctness of a control algorithm depends on the dynamic response of the system it controls.

This creates a gap: IPS control software cannot be fully verified without access to physical hardware.

1.2.3 Framework Requirements

A lightweight and flexible framework is needed. This refers to a solution with low computational and minimal infrastructure requirements. Enabling real time execution on standard equipment with low latency (target cycle times$\leq$ 10ms). The virtual plant model should produce I/O signals with dynamics that are sufficiently close to real hardware to allow control algorithms to behave as they would in the physical system.

The absence of such a framework currently limits early verification, debugging, and optimization of IPS control applications.

1.2.4 Proposed solution

Figure 1.2 shows the simulation framework proposed and implemented in this thesis.

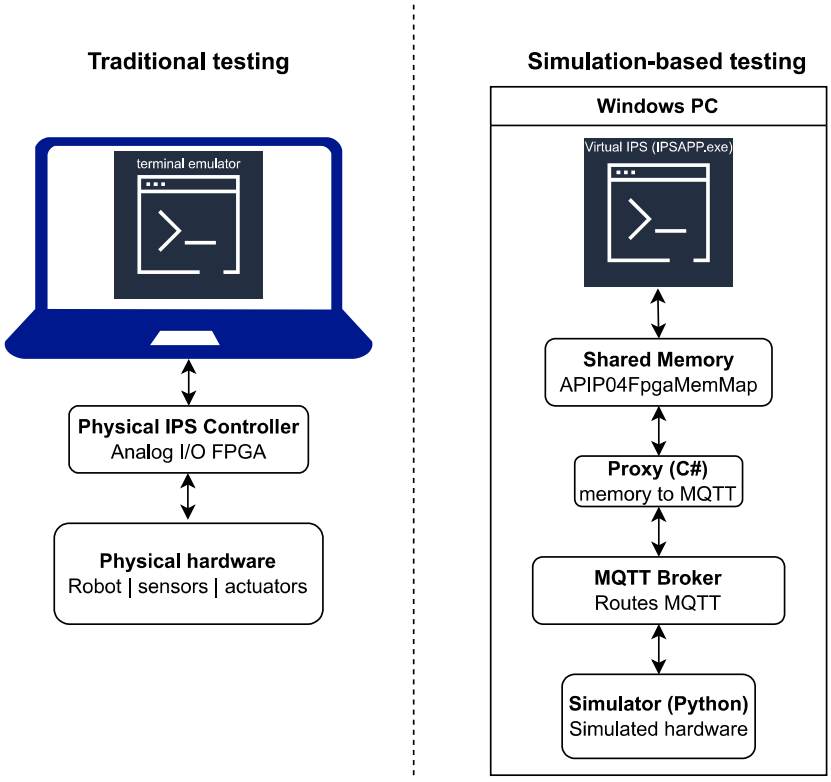


Figure 1.2: Comparison between traditional hardware-based testing (left) and the proposed simulation-based framework (right).

The traditional setup uses FPGA-managed analog I/O to connect the IPS controller directly to physical hardware. The simulation framework replaces the FPGA register interface with shared memory and MQTT communication, enabling closed-loop testing with a virtual plant while keeping the IPS controller software unchanged.

A proxy application, running alongside the controller, continuously reads the controller's output from shared memory and forwards it over MQTT to a software simulator. The simulator models the physical process, computes a response, and returns feedback through the same communication path. From the controller's perspective, the interface is unchanged: it reads and writes the same memory addresses regardless of whether real hardware or a simulator is on the other end.

1.2.5 Delimitations

The following delimitations apply to the scope of this thesis:

- The implementation focuses exclusively on a Windows based development environment. Other operating systems are not considered in this work.
- Full integration with ABB RobotStudio, ABB's PC-based simulation and offline programming tool for industrial robots (see Section 2.5.1), is considered a long term objective and is therefore not implemented as part of this thesis.

1.2.6 Research Questions

This thesis investigates how simulation of low-level I/O signals can be integrated into a virtual environment for IPS. To guide the work, the following research questions are defined:

1. How can MQTT, a protocol for sending messages 2.2.1, can be used to enable communication between a virtual IPS environment and external simulation components?

2. How can shared memory mechanisms be used to exchange low level I/O signals between the IPS controller and external applications?
3. To what extent can virtual signals be simulated in order to reproduce realistic system behavior within a virtual IPS environment?

1.2.7 Goals

The following goals define what the thesis aims to achieve and provide a basis for evaluating the results.

Short-term Goals

1. Implement MQTT-based communication between the virtual IPS environment and an external simulator, with reliable message delivery and a mean round-trip latency ≤ 10 ms.
2. Enable the proxy to read and write analog I/O signals via Windows shared memory within one IPS control cycle (approximately 17 ms).
3. Demonstrate that the simulated hardware reproduces physical hardware behavior with a mean absolute output error below 0.5 V over the $\pm$10 V operating range.

Long-term goals

1. Integrating the Virtual IPS as part of ABB RobotStudio, (See section 2.5.1), to enable combined simulation of process control and robot controller. This integration is considered out of scope of this thesis and is discussed further in section1.2.5.

1.2.8 Choice of Simulated Process

The long-term vision for this framework, as laid out by ABB, involves simulating industrial actuators such as motors, pumps, and paint valves, systems that ex-

hibit nonlinear behavior. Implementing a full motor model within the scope of this thesis introduces significant challenges: It requires power electronics, mechanical load modeling, encoder feedback and calibration, all of which exceed the available time and resources.

An RLC series circuit was therefore chosen as a representative substitute. While it does not capture motor-specific behavior, it is mathematically well-defined and shares several dynamic properties relevant to industrial control systems:

- **Second-order dynamics:** The RLC circuit is represented as a second-order differential equation, producing transient behavior such as rise time and settling time that is representative of many real processes.
- **Analog I/O compatibility:** The circuit maps directly onto the IPS analog interface, the control signal is applied as a voltage via the analog output, and the capacitor voltage is measured through the analog input. This allows closed-loop testing without any modification to the IPS controller.
- **Physical verifiability:** The circuit can be built with standard laboratory components and measured with an oscilloscope, enabling direct comparison between the physical response and the simulation. This makes it possible to validate the framework's accuracy in a controlled manner that would be significantly harder with a motor.
- **Deterministic behavior:** Unlike a motor, the RLC circuit introduces no mechanical nonlinearities such as friction or backlash, making it easier to isolate communication and simulation errors introduced by the simulated hardware.

This substitution reduces how accurately the simulation reflects the long-term goals. The RLC circuit does not capture the behavior of a motor. The later chosen component values 3.1 result in a system that respond slowly without any oscillation, making it a simpler and more predictable system than a real motor. The framework should therefore be regarded as an early proof of concept that demonstrates MQTT communication between a virtual controller and a simulated process, rather than a complete simulation of the ABB painting system.

1.3 Outline

- Chapter 2 presents the selected technologies.
- Chapter 3 presents the hardware platform and the physical RLC circuit used as a representative industrial process.
- Chapter 4 describes the system architecture and key design decisions.
- Chapter 5 introduce the model and its integration into the system.
- Chapter 6 evaluates the framework in terms of latency, accuracy, and robustness.
- Chapter 7 discusses the results.
- Chapter 8 concludes the thesis.

Chapter 2

Technology

This chapter presents the core technologies used in this project and justifies their selection with respect to system requirements and constraints.

2.1 Programming languages

Two programming languages were used in this project, both selected based on the requirements of the component they implement.

2.1.1 Python

Python was used to implement the hardware simulator 5.4. Its dynamic typing and rapid development capabilities enabled efficient implementation and tuning of simulation parameters.

2.1.2 C#

C# was selected for its strong type safety, high performance, and efficient runtime environment. These properties make it well suited for implementing a stable communication layer and meeting the timing requirements of the control loop.

2.2 Protocol

The following messaging protocol forms the communication backbone of the framework

2.2.1 MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol designed for communication between distributed systems and devices. The protocol follows a publish/subscribe communication model, where clients exchange messages through a central broker rather than communicating directly with each other [11]. This architecture enables loosely coupled communication between system components and is widely used in Internet of Things (IoT) applications.

In this project, MQTT serves as the communication layer between simulation components. Control commands, system states, and simulation data are transmitted between modules using MQTT topics, allowing each component to operate independently while still exchanging information in real time interactions.

MQTT Client

An MQTT client is any device or application that connects to an MQTT broker in order to publish or subscribe to messages. Clients publish messages to specific topics on the broker, and other clients can subscribe to these topics to receive the corresponding messages. This mechanism allows multiple components to exchange information without requiring direct connections between them.

MQTT Broker

The MQTT broker acts as the central communication hub in the publish/subscribe architecture. It receives messages from publishing clients and forwards them to all clients subscribed to the relevant topics. By managing message dis-

tribution, the broker simplifies communication between system components and enables scalable and flexible system architectures. MQTT supports three Quality of Service (QoS) levels: 0, 1 and 2, that offers different delivery guarantees against overhead. The level used in this project is discussed in section 4.1.3

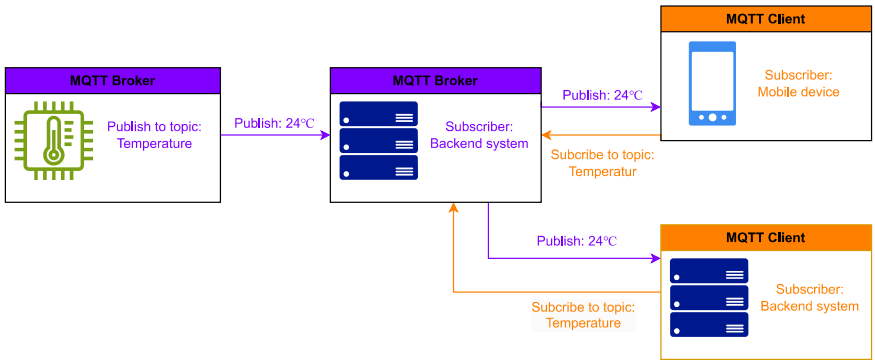


Figure 2.1: MQTT example architecture.

MQTT Message Structure

- **Topic:** The channel in which the message is sent.
- **Payload:** The message/data.
- **QOS:** Quality of service. Determines the reliability of the payload. Balances reliability and resource usage.

2.3 Frameworks

The following libraries were used to implement MQTT communication in each respective language.

2.3.1 Python paho-MQTT

Paho MQTT is an open-source client library provided by the Eclipse Foundation for connecting Python applications to MQTT brokers. It handles connection management, topic subscription, message publishing, and QoS handling.[6]

2.3.2 C# MQTTnet

MQTTnet is an open-source MQTT client library for .NET [5]. It provides a high-performance asynchronous API for connecting to brokers, publishing messages, and handling incoming messages. In this project, it is used to connect the shared memory interface with the MQTT communication layer.

2.4 Data formats

Messages exchanged between framework components are structured using the following format.

2.4.1 JSON

JSON (JavaScript Object Notation) is a lightweight, text-based format for structured data exchange [8]. It represents data using key-value pairs, making it both human-readable and easy to parse.

JSON was selected due to its simplicity, widespread adoption in IoT systems, and strong support across programming languages.

2.5 Development tools and runtime environments

This section describes the software tools used during development and testing of the framework.

2.5.1 ABB RobotStudio

ABB RobotStudio is a PC-based simulation and offline programming tool for ABB industrial robots [3]. It allows engineers to create virtual robot cells, develop and test robot programs, and validate system behavior without requiring access to physical hardware.

In the context of this thesis, RobotStudio is relevant as a long-term integration target. The goal is to eventually connect the Virtual IPS environment with RobotStudio, enabling combined simulation of both the process control layer and the robot motion controller. This integration is considered out of scope for the current work and is discussed further in Section 1.2.7.

2.5.2 Tera Term

Tera Term is a free and open-source terminal emulator for Windows [13], primarily used in this project for serial communication (UART/RS-232). It provides command-line interface for manually sending commands to the IPS board and viewing serial data received over RS-232

The tool was chosen for its simplicity, low resource usage, and support for serial communication.

Chapter 3

Hardware

This chapter presents the physical hardware components used in the simulation framework. Understanding the hardware is necessary because the framework is designed to reproduce the hardware behavior in software while preserving the same interfaces and constrains. The chapter describes the IPS APIP-04 embedded controller, the analog I/O interface used for communication, and the series RLC circuit used as the representative industrial process. It also documents the physical test setup and measured responses used later for validation in chapter 6.

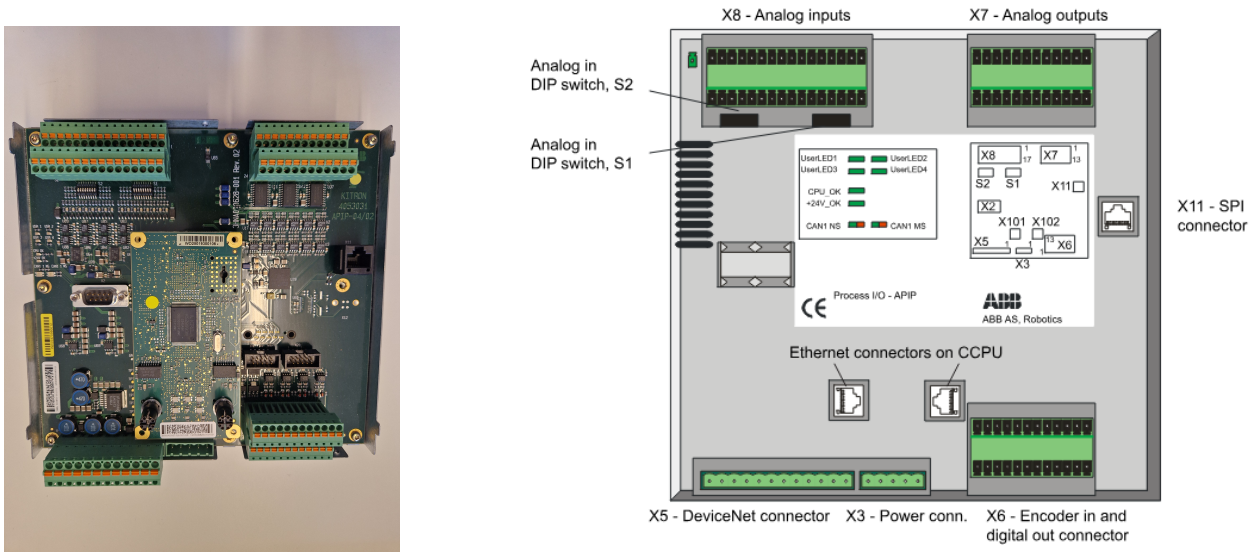
3.1 IPS APIP-04

The IPS APIP-04, developed by ABB, is an embedded industrial control unit for deterministic process control in industrial painting systems. The controller executes its control logic in a fixed cyclic schedule of approximately 17 ms, ensuring predictable sampling and actuation timing. This deterministic behaviour is important for the virtualization framework, since the proxy and MQTT communication layers must operate within strict timing constraints to avoid distorting the original control behaviour.

The APIP-04 interfaces with external systems through analog and digital I/O, where its primary tasks are to acquire sensor data, compute control actions, and issue actuator commands. Analog signal handling is managed by an FPGA re-

sponsible for analog-to-digital and digital-to-analog conversion, using 16-bit unsigned offset binary encoding over a ±10 V range. This yields a voltage resolution of approximately 0.305 mV per LSB (20 V / 65535), which is well within the precision requirements for the RLC validation in this work. The steady state deviations of 0.02 V were observed between AI1 and the oscilloscope. The board provides multiple analog input and output channels accessible through connectors X8 (analog inputs) and X7 (analog outputs), as shown in Figure 3.1.

Figure 3.1 shows the IPS APIP-04 board and its schematic diagram.



(a) IPS APIP-04

(b) IPS APIP-04 Schematic

Figure 3.1: IPS APIP-04 hardware and schematic diagram. Analog input channels are accessible via connector X8, and analog output channels via connector X7.

The controller operates with a fixed control cycle of approximately 17 ms and supports analog signals in the range of $\pm 10\,\mathrm{V}$. This enables deterministic sampling and actuation, which is essential for closed-loop control.

3.2 IPS Hardware Interface

The APIP-04 exposes its I/O functionality through two main interfaces: an analog input and output interface for interacting with sensors and actuators, and a serial terminal interface for configuration and monitoring. Both interfaces are used in this project and are described in the following subsections.

3.2.1 Analog Input and Output Handling

The IPS controller provides analog input and output interfaces for interaction with external hardware. In a closed-loop configuration, the analog output represents the control signal applied to the process, while the analog input provides feedback used by the controller to compute subsequent control actions. Outputs can be configured to generate voltage or current signals representing control commands, while the inputs are used to measure corresponding signals from the system.

In this project, the analog input channel is used to obtain feedback from the process. In a physical setup, this would correspond to measuring a real signal from hardware.

The use of analog I/O enables the controller to operate in a manner consistent with real-world industrial systems, with all signals configured and accessed through the IPS interface, providing a uniform mechanism for writing outputs and reading inputs in both physical and simulated environments.

Although the APIP-04 platform supports multiple analog I/O channels, this project virtualize a single analog output and a single analog input channel (voltage mode) in order to validate the proposed architecture while limiting system complexity.

3.2.2 Terminal interface and commands

The IPS board is accessed from a computer via a serial (RS-232) terminal interface using a terminal emulator (Tera Term 2.5.2), which enables configuration, monitoring, and manual interaction with the system.

The terminal interface supports manual reading and writing of analog signals, but does not provide automated data logging. This limits the ability to capture time-series data during dynamic tests, and is discussed further in Section 6.2.

For instructions on establishing communication, configuring the terminal, and using IPS commands for reading and writing analog signals, see Appendix A.

3.3 Representing an industrial process

To evaluate the IPS controller in a closed-loop configuration, a representative industrial process model is required. This model enables direct comparison between a physical system and its digital simulation, allowing validation of the simulation framework and assessment of how accurately it reproduces real system dynamics.

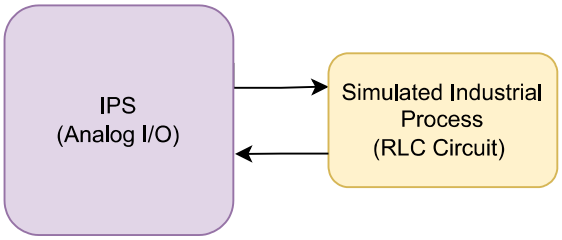


Figure 3.2: IPS and simulated industrial process

3.3.1 RLC Circuit as Representative Industrial Process

A second-order RLC circuit was selected as the representative process due to its ability to capture key dynamic behaviors relevant to industrial control systems. Compared to first-order systems (e.g. RC circuits), it exhibits richer dynamics, including energy storage, damping, and potential oscillatory responses.

The circuit maps directly to the analog input/output structure of the IPS con-

troller. The control signal is applied as a voltage via the analog output, while the resulting capacitor voltage is measured through the analog input. This forms a standard closed-loop configuration, enabling realistic validation of signal acquisition, control execution, and feedback handling within the IPS environment.

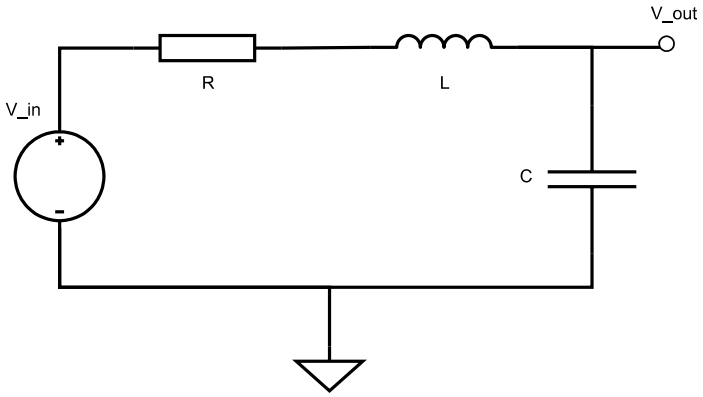


Figure 3.3: RLC circuit overview

3.3.2 Component Selection and Transfer Function

Component values were chosen to respect the IPS analog output current limit ($\approx 5\,\text{mA}$ at $\pm 10\,\text{V}$), requiring a minimum load resistance of $2\,\text{k}\Omega$. Although oscillatory behavior is theoretically possible in second-order RLC systems, the selected component values intentionally produce an overdamped response to ensure stable behavior and compatibility with the IPS and MQTT timing constraints. The high series resistance in turn necessitates correspondingly large L and C values to keep the plant dynamics slow enough to be accurately captured by both the IPS control cycle and the MQTT polling interval of the digital system. The selected values are summarised in Table 3.1.

Table 3.1: Selected component values for the RLC circuit

Component	Value	Justification
Resistor R	3 kΩ	Current limitation for IPS (max 5 mA at 10 V)
Inductor L	2 H	Time constant constraints
Capacitor C	1000 μF	Time constant constraints

It should be noted that the effective resistance includes the internal resistance of the inductor, resulting in a total series resistance of approximately 3 kΩ.

The series RLC transfer function from input voltage V_{in} to capacitor voltage V_{out} is

$$G(s) = \frac{1}{LCs^2 + RCs + 1}.$$

With $R = 3\,\text{k}\Omega$, $L = 2\,\text{H}$, $C = 1000\,\mu\text{F}$, the system is heavily overdamped ($\zeta \approx 33.54$), with a dominant time constant $\tau_{\text{dom}} = RC = 3\,\text{s}$ and a settling time of approximately $12\,\text{s}$, more than 100 times slower than both the IPS control cycle ($\approx 17\,\text{ms}$) and the MQTT polling interval ($25\,\text{ms}$), ensuring the full response is captured without aliasing.

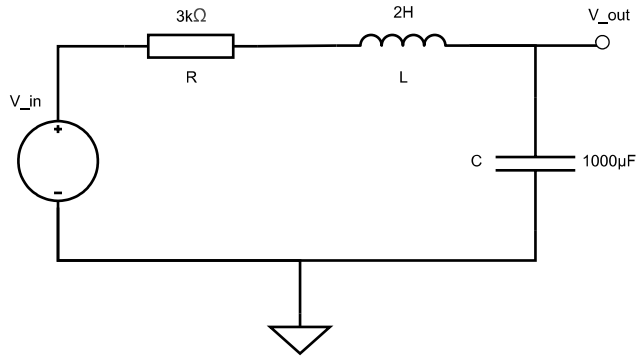


Figure 3.4: Series RLC circuit used for testing. The input voltage V_{in} drives the circuit and the output voltage V_{out} is measured across the capacitor.

3.4 Physical Setup

The physical setup is shown in Figure 3.6, with wiring detailed in Figure 3.5. It consists of the following equipment:

- PL303QMD-P Quad-Mode Dual Power Supply 30V 3A/6A LXI
- Project Boards model GL-23
- Brainboxes US-101 USB-to-RS232 Serial Adapter
- RS232 Serial Cable
- Keysight InfiniiVision DSOX3054T Digital Storage Oscilloscope
- CEI1010 PoI2 CATIII 1000V L70 Variable Inductance Decade Box
- basic electrical components: Resistor, Capacitor

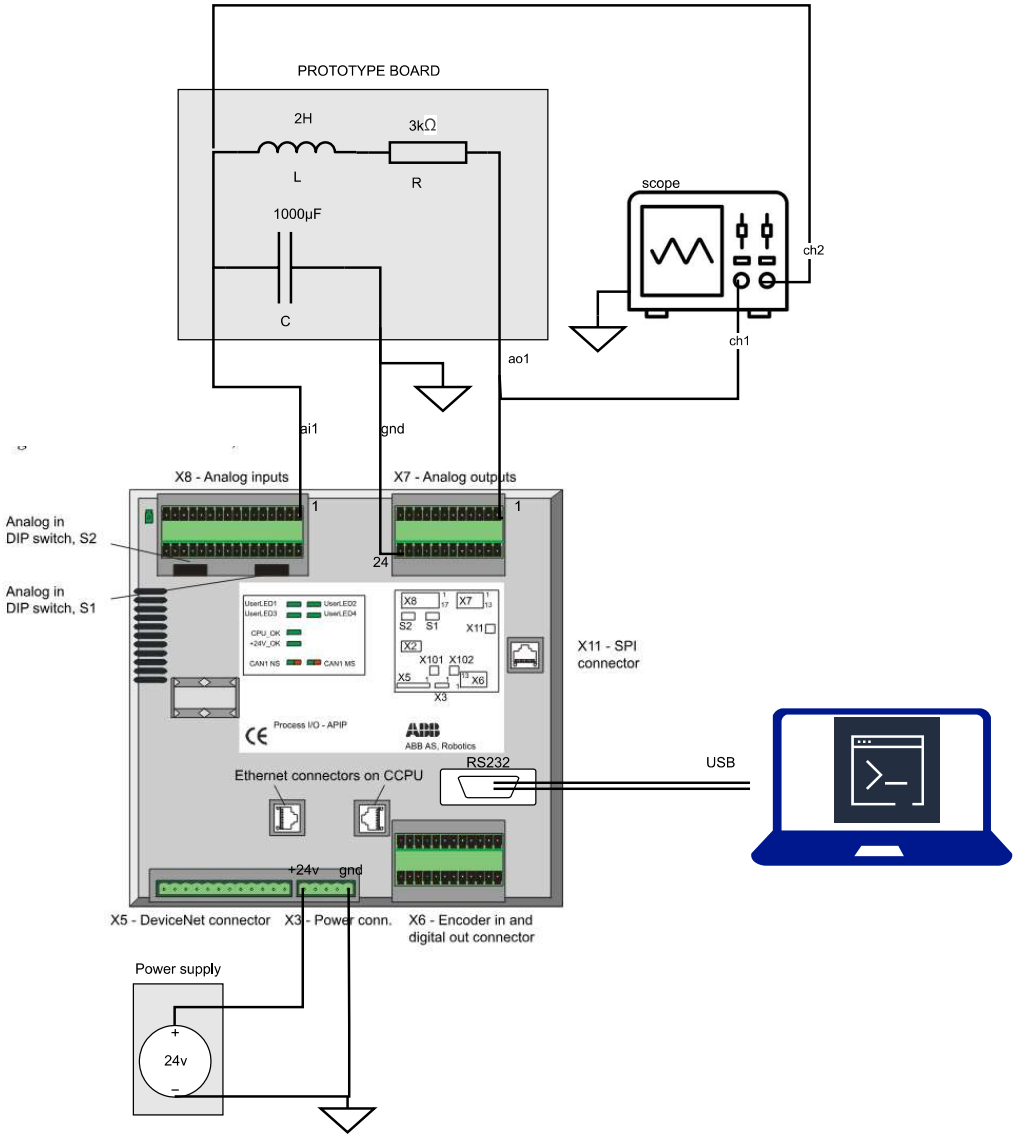


Figure 3.5: Wiring diagram of the physical test setup, showing the connections between the IPS APIP-04 board, the series RLC circuit, and supporting measurement equipment.

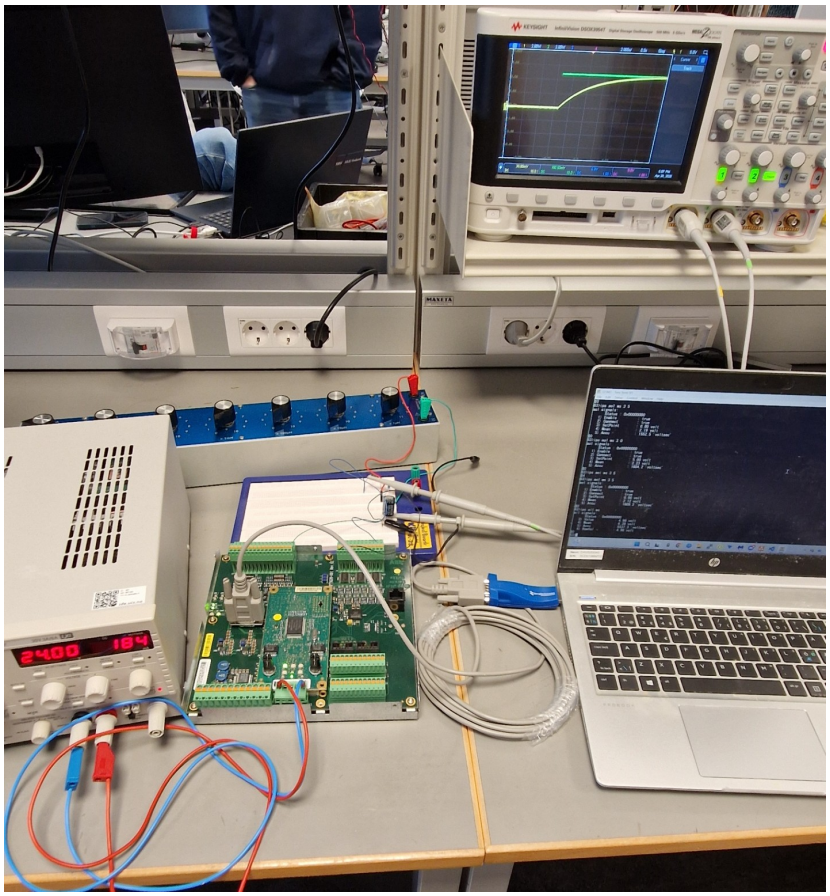


Figure 3.6: Physical hardware setup consisting of the PL303QMD-P power supply, the APIP-04 circuit board, and the Brainboxes RS232 serial communication interface.

Communication between the APIP-04 board and the host computer is established via RS-232 using the Brainboxes USB-to-serial adapter, enabling monitoring and interaction through the IPS terminal.

3.4.1 Test Procedure

The analog output was manually stepped using IPS terminal commands, setting channel AO1 to 0 V, 5 V, and 10 V in sequence (see Appendix A for command reference):

```
63> ips ao1 ws 0
63> ips ao1 ws 5
63> ips ao1 ws 10
```

Step inputs were applied as a standard method for evaluating transient response characteristics, including rise time, settling time, and steady-state behavior. The resulting responses were used to assess the dynamic behavior of the RLC circuit and to verify consistency with the expected model.

3.4.2 Measured Response

Figure 3.7 shows the oscilloscope response to the step inputs, where V_{out} across the capacitor corresponds directly to the output signal in the RLC model.

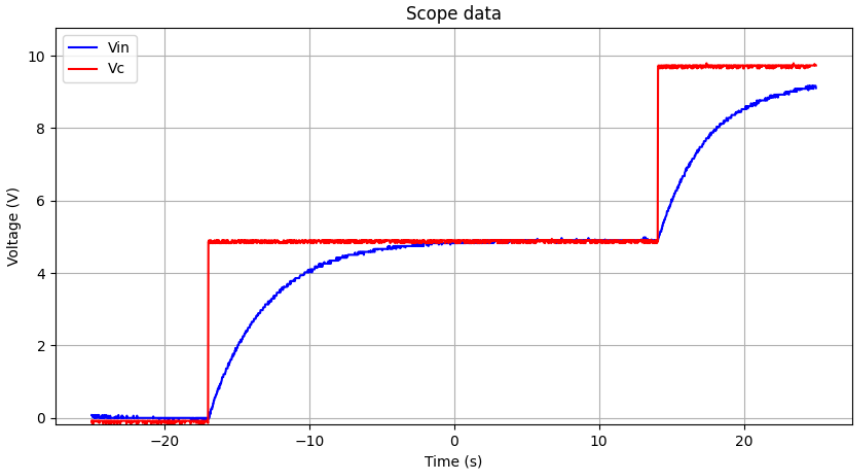


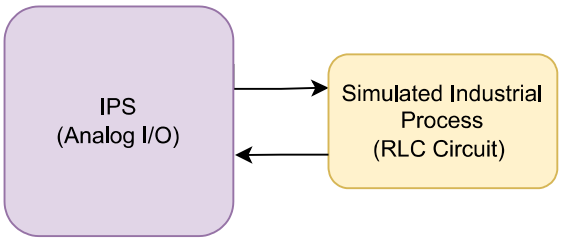
Figure 3.7: Measured hardware response to step inputs of 0 V, 5 V, and 10 V on analog output channel AO1.

The measured responses confirm the expected overdamped behavior, with a settling time of approximately 12,s, consistent with the theoretical model. These results serve as the physical reference against the simulation framework which is validated in Chapter 6.2.3.

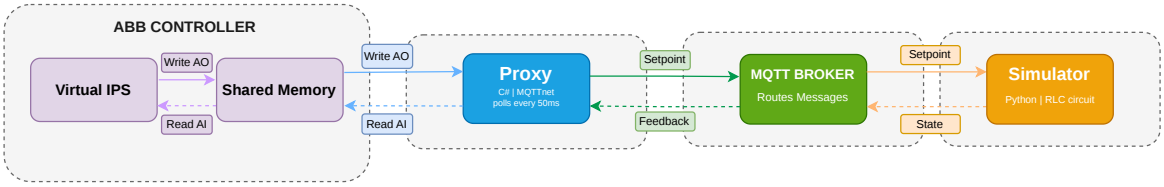
Chapter 4

Framework design

This chapter describes the design of the simulation framework used to replace the physical IPS setup with a software-based environment. The framework consists of three main components: the Virual IPS controller, a C# proxy, and an MQTT broker. Figure 4.1 compares the physical setup with the simulated architecture.



(a) IPS and industrial process (RLC circuit)



(b) Simulated system architecture

Figure 4.1: Comparison between IPS-based signal interface and the simulated system architecture

The following sections describe each component and the reason behind the main implementation choices.

4.1 Components

The framework consists of three main components shown in figure 4.1b:

- ABB CPU Controller (IPS)
- Proxy
- MQTT broker

Below is an explanation of what they are and how they work.

4.1.1 ABB CPU Controller

The controller used in this project is ABB's IPS, a process controller written in C++ and developed by ABB in Bryne. The IPS software is designed for ABB's APIP-04 hardware platform. The same program can be compiled for both the target hardware and for Windows, enabling it to run on a physical target or in simulation mode without any physical hardware attached.

Shared Memory

On target hardware, the IPS controller accesses I/O through the physical memory addresses mapped to the FPGA. In the original version of the software, this memory was allocated privately using `calloc`, making it inaccessible to external processes:

```
1 map_base = calloc(num, size);
```

To enable integration with our proxy component, ABB modified the memory-mapping layer to use Windows named shared memory instead.

```
1 map_base = CreateSharedMem(pageSize, name);
```

The `CreateSharedMem` function uses the Windows API to create a file mapping backed by the system paging file and returns a mapped view of the region:

```
HANDLE hMapFile = CreateFileMappingA(
    INVALID_HANDLE_VALUE,
    NULL,
    PAGE_READWRITE,
    0,
    (DWORD)pageSize,
    name ? name : "Apip04FpgaMemMap"
);
map_base = MapViewOfFile(
    hMapFile,
    FILE_MAP_ALL_ACCESS,
    0, 0,
    pageSize
);
```

This creates a named shared memory region `Apip04FpgaMemMap` that any process on the same machine can open. From the controller's perspective, the memory behaves identically to the FPGA registers on the real hardware. It reads and writes the same offsets using the same code paths. The only difference is that the storage is a shared memory segment rather than a physical device. In the Windows simulation, the FPGA register interface is therefore replaced by this shared memory region.

Unlike the deterministic FPGA-based I/O on the target platform, this Windows simulation introduces non-deterministic timing due to shared memory and asynchronous MQTT communication, a factor evaluated in Sections 6.1 and 6.2.

When the controller starts, it creates the shared memory region and initializes its subsystems. The analog I/O channels are created by loading a configuration file through the IPS shell:

```
22> cfg load ips.cfg
loading config file 'ips.cfg'
Created Anout1 at address 0x00AE0200
Created Anin1 at address 0x00AE0100
 -> created 2 item(s)
```

The shared memory region is named `Apip04FpgaMemMap`. The I/O offsets are computed relative to the base address of this region: the analog output (`Anout1`) is at offset `0x200` , and the analog input (`Anin1`) at offset `0x100`. Both channels store their values as 16-bit unsigned integers in offset binary encoding, where the midpoint is `0x8000` , which represents 0 V and the full range maps to ± 10 V.

Virtual analog I/O Interface

The controller exposes two I/O channels through the shared memory. The analog output (`ao1`) contains a `SetPoint` signal representing the control output, which the controller writes to offset `0x200`. The analog input (`ai1`) at offset `0x100` is intended to provide feedback from the controlled process.

The IPS shell provides commands for inspecting and writing signals. For example, writing a setpoint of 4 V to the analog output:

```
22> ips ao1 ws 3 4
ao1 signals:
      Status : 0x00000000
   1) Enable    : true
   2) Connect   : true
   3) SetPoint  : 4.00 volt
   4) Mean      : 0.01 volt
   5) Accu      : 7.2 'voltsec'
```

The `ips <device> ws` command displays all signals for a device, the third argument selects a specific signal by index and the fourth provides the value to write. Table 4.1 lists the signals and parameters exposed by the `ANOUT` device type.

Memory Encoding

Values in the shared memory are stored as 16-bit integers. The analog input/output uses an unsigned encoding with a midpoint offset of 32768, mapping the ± 10 V

Table 4.1: ANOUT signals and parameters (from IPS Reference Manual [4].

Signal	Access	Type	Description
Enable	r/w	bool	Enable/disable the analog output
SetPoint	r/w	real [V]	Desired output voltage
Mean	r	real [V]	Measured mean output value
Accu	r/w	real [Vs]	Accumulated output (consumption)

range to $[0,\ 65535]$:

$$V = \frac{(\texttt{raw} - 32768) \times 10}{32767}, \qquad \texttt{raw} = \left\lfloor \frac{V \times 32767}{10} + 32768 \right\rfloor \tag{4.1}$$

Table 4.2 shows representative encoding values for both channels.

Table 4.2: Example voltage-to-raw encoding for analog I/O channels.

Voltage [V]	Analog
-10.0	`0x0000` (0)
0.0	`0x8000` (32768)
5.0	`0xBFFE` (49150)
10.0	`0xFFFF` (65535)

4.1.2 Proxy

The proxy is a C# application that connects the IPS controller's shared memory with MQTT. It reads the analog output register written by the controller and publishes the value to a topic, making the controller's setpoint available to any subscriber. The proxy is structured into two components, a shared memory and an MQTT publisher.

Shared Memory

The `Memory` class encapsulates all access to the controller's shared memory region. It opens the named memory segment `Apip04FpgaMemMap` created by the IPS controller and maps a read/write view of the region:

```
private readonly MemoryMappedFile _mmf;
private readonly MemoryMappedViewAccessor _accessor;
public Memory()
{
    _mmf = MemoryMappedFile.OpenExisting(_name);
    _accessor = _mmf.CreateViewAccessor(0, _size,
        MemoryMappedFileAccess.Read);
}
```

The class exposes a single public method, `ReadAnalogOutput`, which reads the 16-bit register at offset `0x200` and converts it from offset binary encoding to a voltage value:

```
public double ReadAnalogOutput()
{
    ushort raw = _accessor.ReadUInt16(_anout1Offset);
    return FromRegister(raw);
}
```

The register uses offset binary encoding, a standard representation for bipolar analog signals in 16-bit registers. The midpoint value `0x8000` (32768) represents

0 V, and the full unsigned range maps linearly to ± 10 V. The conversion is:

$$V = \frac{(raw - 32768) \times 10}{32767} \tag{4.2}$$

MQTT Bridge

The `MqttBridge` class manages the connection to the broker and provides a method for publishing setpoint values. It uses the MQTTnet library and connects with a clean session to ensure no state from previous runs.

```
    public static async Task<MqttBridge> ConnectAsync(
        string broker = "localhost", int port = 1883,
        string clientId = "ABB_PROXY",
        string pubTopic = "circuit/setpoint/cmd",
        string subTopic = "circuit/state/value")
    {
        var factory = new MqttClientFactory();
        var client = factory.CreateMqttClient();

        var options = new MqttClientOptionsBuilder()
            .WithTcpServer(broker, port)
            .WithClientId(clientId)
            .WithCleanSession(true)
            .Build();

        await client.ConnectAsync(options);
        Console.WriteLine($"Connected to MQTT broker at {
            broker}:{port}");

        return new MqttBridge(client, pubTopic, subTopic);
    }
```

Messages are published with the retain flag set, so the broker stores the latest setpoint. This ensures that subscribers connecting after the value was published still receive the most recent setpoint immediately, rather than waiting for the next change.

Main Loop

Every 25 ms, it reads the analog output register and publishes the value to the broker if the value has changed. The 25 ms interval was chosen as a trade-off: the IPS control cycle runs at approximately 17 ms, so the proxy may observe a setpoint only after up to three control cycles have elapsed. The simulated RLC plant has a time constant of 3 s and a settling time of approximately 12 s, meaning that changes missed within a single 25 ms window have negligible effect on the plant response. A shorter polling interval would reduce this lag but increase CPU usage and MQTT message rate without significantly improving simulation accuracy for this plant.

```
using var memory = new Memory();
await using var mqtt = await MqttPublisher.ConnectAsync();
double lastSetpoint = double.NaN;
while (true)
{
    double setpoint = memory.ReadAnalogOutput();
    if (setpoint != lastSetpoint)
    {
        lastSetpoint = setpoint;
        await mqtt.PublishAsync(setpoint);
    }
    await Task.Delay(PollMs);
}
```

The setpoint value is only sent to the broker when it changes. This is done to prevent the broker from storing duplicate messages.

4.1.3 MQTT Broker

The broker is a service that runs in the background. It receives the setpoint from the client and forwards it to the simulator.

The broker manages two active topics in this system, shown in Table 4.3.

Table 4.3: MQTT topics used in the framework

Topic	Publisher	Subscribers	QoS
`circuit/setpoint/cmd`	Client	Simulator	1
`circuit/state/value`	Simulator	Client	1

Quality of Service

Both topics use Quality of Service level 1 (At Least Once), which guarantees that each message is delivered at least once. At this level, the broker stores the message until the receiving client acknowledges it with a PUBACK packet. This was chosen as a compromise between reliability and latency: QoS 0 (At Most Once) offers no delivery guaranty, while QoS 2 (Exactly Once) introduces a four-step handshake that adds overhead that is unsuitable for a real-time control loop.[7] The latency difference in Quality of Service was tested in chapter 6.1.1.

Chapter 5

RLC Circuit Model

5.1 Introduction

The goal of this chapter is to build a virtual representation of the series RLC circuit shown in Figure 3.4, so that the simulator responds to control signals in the same way as the physical hardware. The input signal V_{in} is received as a setpoint via MQTT from the proxy, and the output voltage V_{out} across the capacitor is returned as feedback.

The model captures the second-order dynamic behavior of the physical circuit, including transient response and damping. It does not account for nonlinearities, noise, or parasitic effects, and should therefore be regarded as an approximation sufficient for validating the MQTT communication layer rather than an exact replica of the hardware.

5.1.1 Relation to the Simulation Framework

In the proposed simulation framework, the physical hardware shown in Figure 3.6 is replaced by the RLC simulator. The input voltage is provided through MQTT messages instead of analog output channel, and the output response is computed numerically rather than measured from hardware.

This allows the controller to interact with a virtual system in the same way as it would with real hardware, enabling testing and development without requiring access to physical devices.

5.2 Mathematical Model

The dynamics of the RLC circuit can be described using Kirchhoff's voltage law.[9]. The sum of the voltage drops across the resistor, inductor, and capacitor must equal the input voltage.

$$V_{in} = V_R + V_L + V_C \tag{5.1}$$

There are also the individual component voltages.

$$V_R = Ri \tag{5.2}$$

$$V_L = L\frac{di}{dt} \tag{5.3}$$

$$V_C = v_c \tag{5.4}$$

When combining these expressions, it gets us to the system equation:

$$V_{in} = Ri + L\frac{di}{dt} + v_c \tag{5.5}$$

The capacitor voltage is related to the current through:

$$\frac{dv_c}{dt} = \frac{i}{C} \tag{5.6}$$

These equations form a system of first-order differential equations that describe the dynamic behavior of the circuit.

5.3 Numerical Simulation

To simulate the circuit behavior, the continuous time equations are approximated numerically. The simulation uses a fixed timestep method where the system states are updated iteratively.

The state variables used in the simulation are:

- Capacitor voltage v_c
- Inductor current i_l

The derivatives are:

$$\frac{dv_c}{dt} = \frac{i_l}{c} \tag{5.7}$$

$$\frac{di_l}{dt} = \frac{V_{in} - v_c - Ri_l}{L} \tag{5.8}$$

The simulator updates the states using a forward Euler integration method with timestep Δt.

$$v_c(t + \Delta t) = v_c(t) + \frac{dv_c}{dt}\Delta t \tag{5.9}$$

$$i_l(t + \Delta t) = i_l(t) + \frac{di_l}{dt}\Delta t \tag{5.10}$$

The numerical simulation of the RLC circuit is implemented in Python. The simulator maintains two state variables: the capacitor voltage v_c and the inductor current i_l. At each timestep, the derivatives of these states are computed according to the circuit equations.

5.1 shows the implementation of the state update in the simulator.

Listing 5.1: State update of RLC

```
dv_c = i_l / C
di_l = (v_in - v_c - R * i_l) / L
```

```
3
4 v_c += dv_c * DT
5 i_l += di_l * DT
```

This approach allows the circuit response to be computed in real time while remaining computationally simple.

5.4 Python Implementation

The simulator is implemented as a standalone Python program using the `paho-mqtt` client library 2.3.1. It connects to the broker defined in 4.1.3, subscribes to the setpoint topic, run the integration loop continuously, and publishes the capacitor voltage to the state topic.

5.4.1 Receiving data

The simulator subscribes to the setpoint topic `circuit/setpoint/cmd`. Incoming messages are in JSON format of the form `{"setpoint": <float>}`, and the `on_message` callback parses the payload and updates the input voltage used by the integration loop.

Listing 5.2: MQTT setpoint callback.

```
def on_message(client, userdata, msg):
    global v_in
    try:
        data = json.loads(msg.payload.decode())
        v_in = float(data["setpoint"])
    except json.JSONDecodeError as e:
        print(f"Invalid JSON: {e}")
    except (KeyError, ValueError) as e:
        print(f"Invalid command: {e}")
```

5.4.2 Main loop

The main loop integrates the circuit equations every Δt and publishes the state at a fixed interval. Separating the publish rate from the integration rate keeps the physics step small, which preserves numerical accuracy, while limiting message rate.

Listing 5.3: Integration and publish loop.

```
while True:
    dv_c = i_l / C
    di_l = (v_in - v_c - R * i_l) / L

    v_c += dv_c * DT
    i_l += di_l * DT
    sim_time += DT

    time_since_publish += DT
    if time_since_publish >= PUBLISH_INTERVAL:
        payload = json.dumps({
            "time":     round(sim_time, 2),
            "value":    round(v_c, 4),
            "setpoint": v_in,
        })
        if _connected:
            client.publish(STATE_TOPIC, payload, qos=1)
        time_since_publish = 0.0
```

When the broker is unreachable, the simulation still runs, but the data is not published. This design choice is described in detail Section 6.3.1.

5.5 Simulation Results

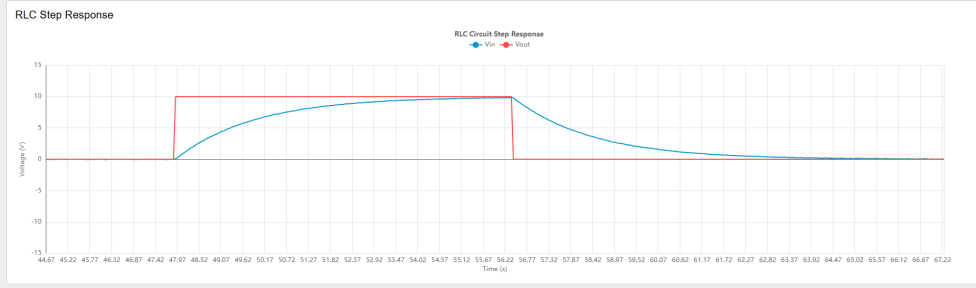


Figure 5.1: Step response of the simulated RLC circuit

Figure 5.1 shows the step response of the simulated RLC circuit. The red curve represents the input voltage V_{in} applied to the circuit, while the blue curve shows the resulting output voltage V_{out} measured across the capacitor.

When the input voltage steps from 0 V to 10 V, the circuit shows a transient response before settling near the steady state value. This behavior is typical for second-order systems.

The observed response demonstrates that the simulator correctly reproduces the dynamic behavior expected from a second-order RLC system.

Chapter 6

Testing and Validation

6.1 Latency

6.1.1 Round-trip time

The latency can be measured externally without modifying any existing code. RTT represents the total time required for a message to travel from a sender, through the MQTT broker and simulation components, and back to the sender as a response.

The round-trip time is relevant in this project because the system is based on distributed components communicating asynchronously through MQTT. In such architectures, latency affects how quickly changes in input signals are reflected in the system output. High or unstable latency may reduce the accuracy of the simulation and limit its usefulness for testing control logic.

A low and consistent RTT indicates that the framework is capable of supporting real-time interactions between the controller and the simulated system.

Method

A python script is used to connect to the broker as an independent client. It records a timestamp at the time of publishing and a second timestamp when the state message is received. The difference is the round-trip time for that sample.

Listing 6.1: Measurement loop

```
send_time = time.perf_counter()
client.publish(SETPOINT_TOPIC,
               json.dumps({"setpoint": setpoint}), qos=1)
# Block until a state message arrives
msgs = wait_for_response(timeout=5.0)
rtt = time.perf_counter() - send_time
```

A dummy message is sent before the measurement to avoid the overhead from the TCP handshake. The script then collects N samples, changing the setpoint to avoid retained messages.

Listing 6.2: Warm-up before measurement.

```
client.subscribe(STATE_TOPIC, qos=1)
time.sleep(0.5)
# Discard the first exchange
client.publish(SETPOINT_TOPIC,
               json.dumps({"setpoint": 0.0}), qos=1)
time.sleep(0.3)
```

Results

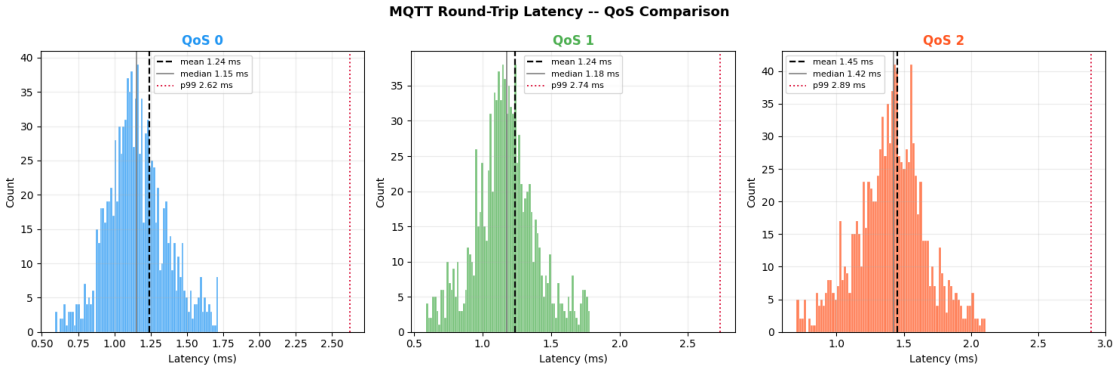


Figure 6.1: Distribution of MQTT round-trip latency for QoS levels 0, 1 and 2

Table 6.1: Key latency metrics for MQTT QoS 0, 1, and 2

QoS Level	Mean [ms]	Median [ms]	p99 [ms]	Delivery Guarantee
QoS 0	1.24	1.15	2.62	At most once
QoS 1	1.24	1.18	2.74	At least once
QoS 2	1.45	1.42	2.89	Exactly once

QoS Comparison

Figure 6.1 and table 6.1 compares the measured round-trip latency for MQTT QoS levels 0, 1, and 2. The results are presented using the mean, median, and 99th percentile (p99) latency. The mean represents the average latency across all samples, the median shows the typical response time, while p99 indicates the worst-case latency experienced by top 1 % of the samples.

QoS 0 provides the lowest latency because messages are transmitted without acknowledgment. It does not guarantee message delivery. QoS 2 introduces the highest latency due to the additional handshake required to guarantee exactly-once delivery.

The framework uses QoS 1, which guarantees at-least-once delivery while maintaining relatively low latency. This level was selected as a compromise between

reliability and performance, since occasional duplicate messages are acceptable for this application, while message loss could affect control validation.

Interpretation

The average latency of 1.24 ms for QOS 1 confirms that the MQTT-based architecture is suitable for real-time control testing. In distributed control systems and simulation environments, a mean RTT below 10 ms is generally considered sufficient for most closed-loop applications such as pressure, flow or motor control [12]. The observed jitter is typical for Python applications running on a non-real-time operating system and does not compromise the validity of the simulation for the purpose of this project.

Overall, the measured latencies indicate that the framework is approved for use in testing IPS control logic. The system can reliably reflect setpoint changes within a few milliseconds, which is more than sufficient for validating regulator performance, delay compensation, and supervision functions.

6.2 Accuracy

The latency evaluation in Section 6.1 focuses on the communication path. In contrast, this section evaluates the accuracy of the framework by comparing the simulated plant response against measurements from the physical hardware setup. The intent is not to show a perfect match, but to quantify how close the simulation is when realistic non-idealities are included.

6.2.1 AI1 Channel Verification

Due to the lack of automated data logging from the IPS terminal, a full time-series comparison between AI1 readings and oscilloscope measurements was not feasible. To partially address this, a single steady-state reading was taken from AI1 via the terminal once the system had settled, and compared against the oscilloscope at the same point in time. At a setpoint of 5 V, the oscilloscope measured

$V_{\text{out}} = 4.91\,\text{V}$ while IPS reported `ai1` $= 4.89\,\text{V}$, giving a deviation of $0.02\,\text{V}$. Although a single sample cannot characterize dynamic accuracy, it suggests that the AI1 channel reads within acceptable bounds at steady state.

6.2.2 Hardware-Simulator Comparison Method

Accuracy is evaluated by applying setpoint steps on the system and logging both the hardware response (oscilloscope capture) and the simulator response. The oscilloscope data is exported as a CSV file and contains three columns: time, measured output voltage, and measured setpoint voltage. A matching simulator dataset is generated on the same time grid.

The comparison is done sample-by-sample on the common $25\,\text{ms}$ time grid in the interval $t \in [-25, 25]\,\text{s}$. For each timestamp t_k, the deviation is computed as

$$e(t_k) = x_{\text{real}}(t_k) - x_{\text{sim}}(t_k), \tag{6.1}$$

where x is either the output voltage V_c or the setpoint/reference voltage V_{in}. The following metrics are reported:

- Mean absolute error (MAE): $\frac{1}{N}\sum_{k=1}^{N} |e(t_k)|$
- Root mean square error (RMSE): $\sqrt{\frac{1}{N}\sum_{k=1}^{N} e(t_k)^2}$
- Maximum absolute error: $\max_k |e(t_k)|$

6.2.3 Results

Figure 6.2 compares the hardware and simulator responses for the setpoint (V_{in}) and output voltage (V_c). The simulator reproduces the overall dynamic behavior, with deviations due to component tolerances, measurement noise, and idealizations. The Hardware setpoints deviates from nominal 5 v (typically 4.9–4.9 v) due to non-idealities in the analog signal chain. In contrast, the simulator uses ideal step inputs, introducing a systemic offset in the setpoint comparison.

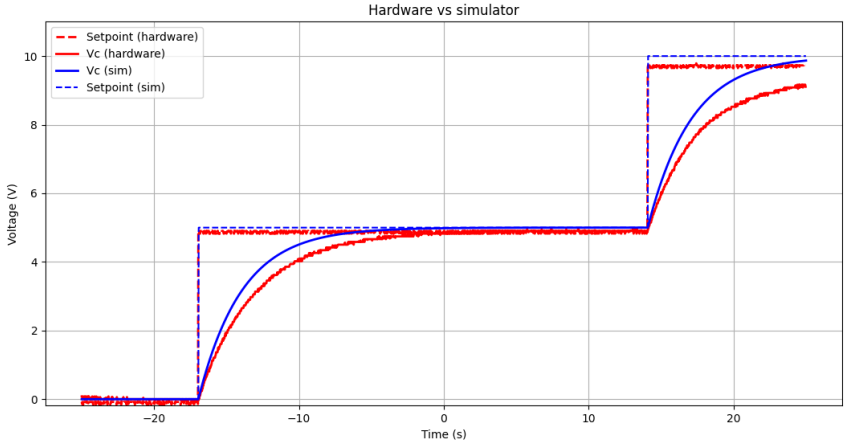


Figure 6.2: Hardware vs simulator response

Table 6.2 summarizes the deviation between the hardware and the simulator signals.

Table 6.2: Deviation between hardware and simulator

Signal	**MAE [V]**	**RMSE [V]**	**Mean err [V]**	**Max |err| [V]**
V_c (output)	0.292	0.391	-0.290	0.850
V_{in} (setpoint)	0.162	0.231	-0.1529	4.864

Discrepancy Between Physical and Simulated Response

The physical system responds more slowly than the simulation predicts. The most likely causes are:

Effective series resistance mismatch.

Although the model uses a nominal resistance of $R = 3\,\text{k}\Omega$, the physical system may still exhibit additional resistance contributions from wiring, contact resistance, variation in the inductor winding resistance and component tolerances.

Any deviation in the effective series resistance directly affects the dominant time constant $\tau = RC$, and therefore the transient response speed. A higher effective resistance in the hardware results in a slower capacitor charging curve, which is consistent with the observed behavior.

IPS output current limitation.

The IPS analog output is rated at approximately 5 mA at $\pm$10 V. With a 3 kΩ load, the steady-state current at 10 V is approximately 3.3 mA, which is below the specified limit.

However, non-ideal output behavior such as internal output impedance and finite current drive capability may still introduce a voltage drop, reducing the effective V_{in} delivered to the circuit. This can result in a slightly slower capacitor charging response compared to the ideal model.

Ideal component values in simulation.

The simulator uses exact nominal values ($R = 3\,\text{k}\Omega$, $L = 2\,\text{H}$, $C = 1000\,\mu\text{F}$), while the physical components have manufacturing tolerances, contact resistance, and parasitic elements that are not modelled. These deviations shift the time constant and transient shape of the physical response away from the theoretical prediction.

Together, these factors produce a systematic offset where the physical circuit charges more slowly than the simulation, which is consistent with the negative mean error of $-0.290\,\text{V}$ reported in Table 6.2.

Output Voltage (V_c)

The output voltage has an MAE of $0.3765\,\text{V}$ and an RMSE of $0.549\,\text{V}$. The negative mean error of $-0.290\,\text{V}$ indicates that the simulator slightly overestimates the capacitor voltage, consistent with a higher effective resistance in the physical circuit due to parasitic effects.

The maximum error $0.850\,\text{V}$ occurs during transients, where the model is sensitive

to parameter mismatch. As the system approaches steady state. the simulator response converge, indicating that the dominant error source is transient mismatch rather than steady-state deviation.

Setpoint Voltage (V_{in})

The setpoint error is smaller $0.162\,\text{V}$, and RMSE of $0.231\,\text{V}$. The large maximum error of $4.864\,\text{V}$ occurs at step transitions to the difference between finite-slope hardware signals and ideal instantaneous steps in the simulator, and does not reflect sustained inaccuracy.

Interpretation

The simulator captures the qualitative dynamics of the physical RLC circuit, including transient behavior and steady-state response. Deviations are primarily caused by component tolerance, parasitic effects, and idealized inputs.

For validating the communication architecture and closed-loop interaction, the accuracy is sufficient. An average output error below $0.5\,\text{V}$, corresponding to approximately $5\,\%$ over a $\pm 10\,\text{V}$ range, is acceptable for early-stage verification.

6.3 Robustness

6.3.1 MQTT Broker failure handling

The broker is a single point of failure, if the broker loses connection, every connected client loses its TCP session.

To fix these concerns, we have implemented: connection-state tracking, automatic reconnection, and continued simulation during outage.

1. **Connection-state tracking.** A `_connected` flag is used by the `on_connect` and `on_disconnect` callback functions. The simulation loop checks this flag

before every publish. This is done to ensure that messages are suppressed rather than queued, preventing a backlog of unsent messages.

Listing 6.3: Connection and disconnection callbacks.

```
_connected = False
def on_connect(client, userdata, flags, reason_code,
    properties):
  global _connected
  if reason_code.is_failure:
      _connected = False
  else:
      _connected = True
      client.subscribe(TARGET_TOPIC)

def on_disconnect(client, userdata, disconnect_flags,
                reason_code, properties):
  global _connected
  _connected = False
```

2. **Automatic reconnection** The client is initialized with `connect_async()` and a configurable reconnect delay `reconnect_delay_set(min_delay=1,`

   When the network loop detects a lost connection it retries automatically, it starts at a one second interval increasing up to ten seconds. This is done to prevent reconnecting too fast.

Listing 6.4: Reconnect delay

```
  client.reconnect_delay_set(min_delay=1, max_delay=10)
  client.connect_async(host, port, keepalive)
```

A keep-alive interval of 30 s is configured in the `connect_async()` 2. If no traffic is exchanged within this period, the client sends a PINGREQ packet to the broker. If no PINGRESP is received, the connection is considered lost and the reconnect cycle begins.

3. **Continued Simulation During Outage** During an outage, the simulation continues to run, only the publication of the messages are paused. This design choice ensures that when the broker restarts, the simulator resumes publishing the current state, instead of historical values.

Listing 6.5: Simulation loop

```
while True:
  # Physics always runs, regardless of broker state
  dv_c = i_l / C
  di_l = (v_in - v_c - R * i_l) / L
  v_c += dv_c * DT
  i_l += di_l * DT
  sim_time += DT
  time_since_publish += DT
  if time_since_publish >= PUBLISH_INTERVAL:
      payload = json.dumps({
          "time": round(sim_time, 2),
          "value": round(v_c + noise, 4),
          "setpoint": v_in,
      })
      # Only publish when the broker connection is
          active
      if _connected:
          client.publish(STATE_TOPIC, payload, qos=1)
      time_since_publish = 0.0
  time.sleep(DT)
```

6.3.2 Limitations in Robustness Testing

Robustness testing was limited in scope. We successfully verified that the proxy could detect broker disconnection and attempt reconnection (using MQTTnet's built-in mechanisms). We did not perform systematic fault injection for common real-world scenarios such as:

- Packet loss on the MQTT link
- Network latency spikes or jitter

- Partial failures (e.g. simulator crashes while proxy is running)
- High-frequency setpoint updates leading to queue buildup

These omissions represent a weakness in the validation. In a production virtual testing framework for ABB, robustness against such faults would be critical, especially when the Virtual IPS is expected to run reliably alongside RobotStudio.

Chapter 7

Discussion

7.1 Simulation Realism

The simulated plant in this work is based on a simplified RLC circuit model. While this provides a clear and mathematically tractable representation of a second-order system, it does not fully capture the complexity of real industrial processes such as motors or actuators used in ABB systems.

The RLC circuit reproduces key dynamic characteristics such as damping, and transient response. Real systems typically include nonlinearities, delays, noise, and more complex interactions between components. As a result, the simulation should be considered a proof of concept rather than a fully realistic representation of the IPS environment.

Despite these limitations, the RLC model is sufficient for validating the communication architecture and demonstrating how virtual I/O signals can be exchanged between distributed components.

7.2 Limitations

7.2.1 Broker dependency

The MQTT broker represents a single point of failure, as all communication depends on its availability. Although reconnection mechanisms are implemented 6.3.1, a broker failure will interrupt the entire system.

7.2.2 Platform dependency

The framework is limited to Windows due to its reliance on platform-specific APIs such as `CreateFileMapping` and `MapViewOfFile` 4.1.1, reducing portability.

7.2.3 Simplified simulator model

The simulation model is simplified. The RLC circuit captures key second-order dynamics, but does not represent nonlinearities or hardware-specific behavior, and should therefore be considered a proof of concept.

7.2.4 Asynchronous execution

Finally, the system operates asynchronously without explicit synchronization, which may lead to timing inconsistencies between components.

7.2.5 Substitution of motor model with RLC circuit

The original assignment specifies the development of a simulation framework for embedded systems with MQTT-based I/O virtualization, where a motor model is a central component. The intended motor simulation is expected to represent key actuator dynamics such as speed, acceleration, direction control, and bidirectional feedback between the embedded system and the simulated plant.

A series RLC circuit was selected instead of the specified motor model due to practical constraints. A DC motor system with power electronics, sensing, and mechanical load would require significantly higher implementation complexity, additional safety considerations, and extensive calibration, making it infeasible within the project timeframe.

The RLC circuit provides a compact and low-cost alternative that is easily instrumented and integrates well with the MQTT communication layer. It also exhibits second-order dynamics, including damping and transient response, which allows evaluation of closed-loop control behaviour.

This substitution reduces the fidelity of the simulation relative to the assignment requirements. The RLC system does not represent motor-specific behaviour such as integrative position and velocity dynamics, mechanical nonlinearities (friction, saturation, backlash), or realistic actuator constraints. The strongly overdamped configuration ($\zeta \approx 33.54$) further eliminates oscillatory behaviour, resulting in a significantly simpler dynamic response than that of an industrial motor system.

Consequently, while the implementation fulfills the objective of demonstrating MQTT-based I/O virtualization and bidirectional communication between controller and plant, it does not fully realize the motor simulation requirements defined in the assignment.

7.3 Further improvements

7.3.1 Integration with ABB RobotStudio

Integration with ABB RobotStudio remains a key long-term goal , enabling combined simulation of process and motion control. This integration would allow developers to validate both the IPS control layer and robot motion controller within a single virtual environment, without requiring access to physical hardware.

7.3.2 More realistic simulation Models

Future work should include more realistic plant models, such as motor, pump, or pressure valve simulations, to better reflect the industrial systems described in the long-term vision for this framework. These systems exhibit significantly faster dynamics than the RLC circuit used here.

7.3.3 Integration with External Simulation Tools

Because the simulation layer communicates exclusively through MQTT, any tool capable of connecting to an MQTT broker can act as the simulator. For example, MATLAB/Simulink [10] supports MQTT communication and could replace the Python simulator with a higher-fidelity motor or process model. This openness is a deliberate architectural property of the framework: by keeping the communication layer protocol-based rather than tightly coupled to a specific simulator, the framework can evolve alongside the complexity of the simulated process without requiring changes to the IPS controller or the proxy.

7.3.4 Timing and Synchronization Improvements

Explicit synchronization between the proxy, broker, and simulator would improve timing consistency and reduce the risk of stale setpoints. While the 25 ms proxy polling interval is sufficient for the RLC plant with a 3 s dominant time constant, it is too slow for motor control applications in the ABB roadmap. DC motor speed loops typically operate at 100–500 Hz, requiring 2–10 ms feedback cycles. The current polling interval therefore exceeds a full motor control cycle by an order of magnitude. Supporting such applications would require reducing the polling interval to 1–2 ms and replacing `Task.Delay()` with a high-resolution or event-driven mechanism such as `SetEvent/WaitForSingleObject`. Even then, Windows scheduling jitter may limit suitability for inner-loop motor control without buffering or predictive compensation.

7.3.5 Monitoring and Testing

Additionally, extending the framework with monitoring, logging, and automated testing tools would support more advanced development and validation workflows.

Chapter 8

Conclusion

This work demonstrates that MQTT-based I/O virtualization is a feasible approach for simulation-based testing of embedded systems.

The research questions 1.2.6 has been answered:

1. **MQTT.** The MQTT protocol was used for exchanging messages between the simulated plant and the proxy application. Latency testing in Section 6.1 showed a mean round-trip time of 1.24 ms, remaining below the 10 ms target defined in Section 1.2.7, confirming that the architecture is suitable for closed-loop control testing.

2. **Shared memory.** Replacing locally allocated memory with the shared-memory approach described in Section 4.1.1, enabled the Proxy 4.1.2, to access the IPS memory directly without modifying the IPS controller software. This design provides low-latency read/write access to analog signals within one IPS control cycle, making the connection between the simulator and the IPS both simpler and more reliable.

3. **Virtual I/O realism.** The simulated RLC circuit 3.4 demonstrates a virtual I/O that is used to imitate the reproduce of a key dynamic behavior of a physical hardware system, such as damping. With a mean absolute output error of 0.292 V over a ± 10 V range. This is considered acceptable for early-stage verification, where the primary goal is to validate communica-

tion architecture and closed-loop interaction rather than high-fidelity plant modeling..

Appendix A

IPS Reference

A.1 Communication and Terminal Interface

The IPS hardware is accessed from a host computer via a serial connection using a USB-to-RS232 interface. A terminal emulator, such as Tera Term 2.5.2, is used to establish communication with the controller. This interface enables configuration, monitoring, and manual interaction with the system during operation.

A.1.1 Serial Communication Setup

To establish communication between the PC and the IPS board, the terminal software must be configured with the correct serial parameters. The typical settings are:

- Speed: 115200
- Data bits: 8
- Parity: None
- Stop bits: 1
- Flow control: None

Once connected, the IPS boot sequence is displayed in the terminal. A typical startup sequence is shown in Listing A.1.

Listing A.1: Start sequence for IPS

```
System Init
 - init can1 for mode discovery
   wait for : ucmm get online (mac 63) ................
   wait for : ucmm get online (mac 63) ................
       timeout : not found (0)
 - startup mode : Ethernet
 - ipaddress keep as is (1:1:0)
 - board type APIP-05, 64.00 MB RAM, 396.00 MHz
Ethernet Init
Ips Initialize
 - starting regulation
 - starting supervision
Telnet Server Init
Hit Ctrl-C to skip autoexec.bat
   wait for :  .....
cfg load ips.cfg
loading config file 'ips.cfg' -> created 31 item(s)
($0)
Init User Shell
63>
```

After initialization, the user is presented with a command-line interface where IPS commands can be executed.

A.1.2 Configuration File

The IPS system loads a configuration file (`ips.cfg`) on startup, as seen in the boot sequence above. This file defines and maps the named I/O variables used by the controller, for example assigning `ao1` to a physical analog output channel and `ai1` to a physical analog input channel. These named variables are then used directly in terminal commands and control scripts. Without a valid configuration file, the I/O commands described below will not function correctly.

A.1.3 Analog Input and Output via Terminal

Analog outputs can be written using command-line instructions. For example, the following command sets signal index 3 (SetPoint) of analog output channel `ao1` to 5 V:

```
63> ips ao1 ws 3 5
```

The system responds with status information confirming the applied signal:

```
ao1 signals:
      Status : 0x00000000
   1) Enable          : true
   2) Connect         : true
   3) SetPoint        : 5.00 volt
   4) Mean            : 0.60 volt
   5) Accu            : 306.7 'voltsec'
$0
```

Analog inputs can be read using a similar command:

```
63> ips ai1 ws
```

The returned values represent the measured input signal:

```
ai1 signals:
      Status : 0x00000000
   3) Value           : 10.21 volt
   4) Mean            : 10.21 volt
   5) Accu            : 3717.4 'voltsec'
   6) RawVal          : 10.21 volt
$0
```

Bibliography

[1] ABB: 50 år siden den banebrytende oppfinnelsen av lakkeringsroboten. https://new.abb.com/news/no/detail/36667/50-ar-siden-jaerbuenes-banebrytende-oppfinnelse-av-lakkeringsroboten (2019), accessed: 2026-02-02

[2] ABB: About ABB. https://www.abb.com/global/en/company/about (2026), accessed: 2026-02-02

[3] ABB: RobotStudio. https://new.abb.com/products/robotics/robotstudio (2026), accessed: 2026-05-07

[4] ABB Robotics: Reference Manual: IPS Structure, v4.60. ABB (Oct 2025), accessed: 2026-05-13

[5] dotnet: MQTTnet: A high performance MQTT library for .NET. https://github.com/dotnet/MQTTnet (2026), accessed: 2026-02-23

[6] Eclipse Foundation: Paho MQTT python client. https://pypi.org/project/paho-mqtt/ (2026), accessed: 2026-02-23

[7] HiveMQ: What is MQTT quality of service (QoS) 0, 1, and 2? https://www.hivemq.com/blog/mqtt-essentials-part-6-mqtt-quality-of-service-levels/ (2026), accessed: 2026-02-23

[8] JSON.org: Introducing JSON. https://www.json.org/json-en.html (2026), accessed: 2026-01-29

[9] MathWorks: Analyzing the response of an RLC circuit. https://www.mathworks.com/help/control/ug/analyzing-the-response-of-an-rlc-circuit.html (2026), accessed: 2026-03-16

[10] MathWorks: MATLAB: Programming and numeric computing platform. https://se.mathworks.com/products/matlab.html (2026), accessed: 2026-02-26

[11] MQTT.org: MQTT: The standard for IoT messaging. https://mqtt.org/ (2025), accessed: 2026-01-14

[12] Systems, O.: Understanding network latency and its impact on industrial applications. https://www.omnitron-systems.com/blog/understanding-network-latency-and-its-impact-on-industrial-applications (January 2025), accessed: 2026-05-14

[13] Tera Term Project: Tera term. https://teratermproject.github.io/index-en.html (2026), accessed: 2026-04-14

University of Stavanger

4036 Stavanger
Tel: +47 51 83 10 00
E-mail: post@uis.no
www.uis.no

© 2026 Aleksander Bø, Basel Feras Abu Sablih and Vebjørn Erlandsen